

教師なし学習とクラスタリング

ラベルを失った魚たち

rkskmt

1

前回の振り返り

分類問題ではラベルが必要だった

- 訓練データに「これはサーモン、これはスズキ」という **正解ラベル** が必要
- ロジスティック回帰はラベル付きデータを使って決定境界を学習した

📌 今回の問い

魚にラベルを付ける余裕がない場合、何もできないのか？

2

教師あり学習 vs 教師なし学習

教師あり学習 (Supervised)

- 正解ラベルつきデータを使う
- 「これはサーモン」と教えながら学習
- 例：分類、回帰

教師なし学習 (Unsupervised)

- ラベルなしデータを使う
- データの構造・パターンを自分で見つける
- 例： **クラスタリング**、次元削減

📌 ノート

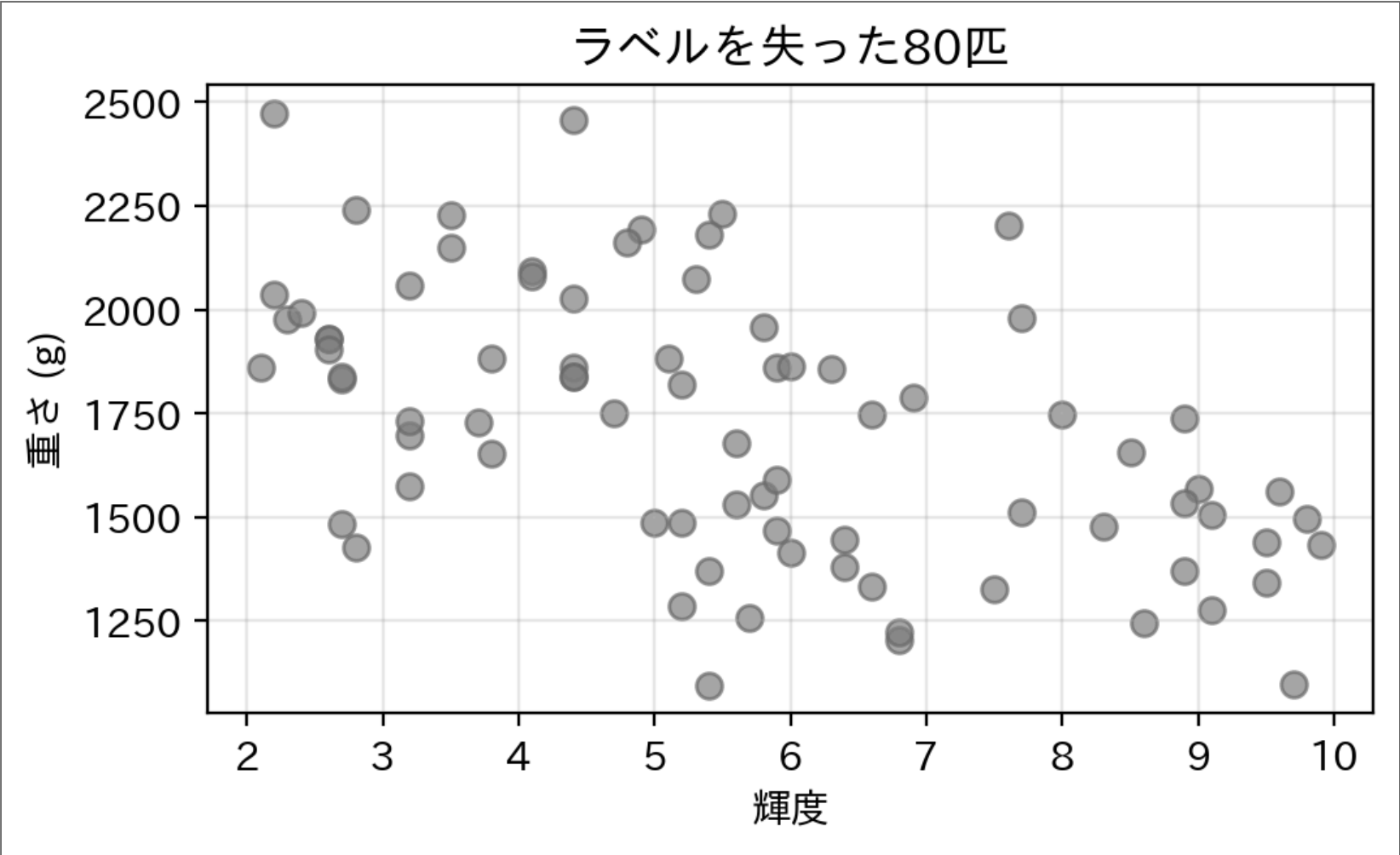
どちらが優れているというわけではなく、 **使えるデータに応じて選ぶ**

3

ラベルを失った魚たち

- 80匹分の測定データ（輝度・重さ）は手元にある
- ところが**どれが Salmon でどれが Sea bass か、記録が消えた**

▶ 魚データの定義（クリックして展開）



❗ クイズ

ラベルなしのこの図、**何個のグループ**に見える？

4

クラスタリングとは

- 人の目には**2つの塊**が見える — では、それを**機械に**見つけさせられるか？
- データを**似たもの同士のグループ（クラスター）**にまとめる
- 正解ラベルがなくても、**データの分布から自然なグループ**を見つける
- 用途の例
 - 顧客をグループに分けて施策を変える
 - 大量の文書を話題ごとに整理する
 - 「どのグループにも入らない点」を見つける（異常検知）

5

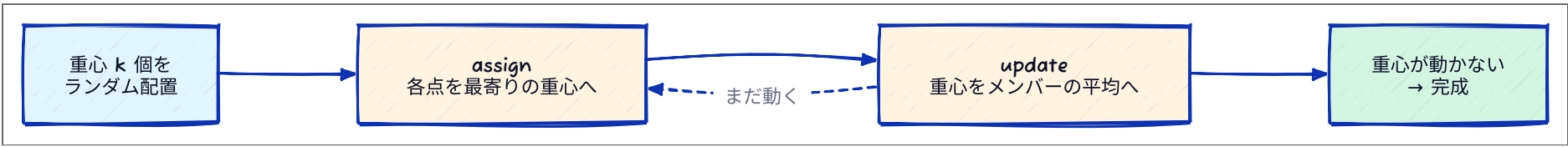
k-means アルゴリズム

6

k-means の手順

入力：データ点の集合、クラスタ数 k

- 1. k 個の重心（centroid）を **ランダム** に配置
- 2. 各データ点を **最も近い重心** のクラスタに割り当てる
- 3. 各クラスタの **重心を再計算**（クラスタ内の平均点）
- 4. 重心が変化しなくなるまで **2〜3** を繰り返す



① ノート

「assign（割り当て）→ update（更新）→ 繰り返し」 たったこれだけのシンプルな構造

7

試行1: そのまま k-means にかける

```
Code
1 from sklearn.cluster import KMeans
2
3 # ラベルを使わずに k=2 でクラスタリング
4 km = KMeans(n_clusters=2, random_state=42, n_init=10)
5 km.fit(X)
6 cluster_labels = km.labels_
7
8 print("クラスタ割り当て（先頭10件）:", cluster_labels[:10])
9 print("重心の座標（輝度，重さ）:")
10 for i, c in enumerate(km.cluster_centers_):
11     print(f" クラスタ {i}: 輝度={c[0]:.2f}, 重さ={c[1]:.0f}g")
```

Result

クラスタ割り当て（先頭10件）： [0 0 0 0 0 0 0 0 0 0]
重心の座標（輝度，重さ）：
 クラスタ 0: 輝度=4.50, 重さ=1977g
 クラスタ 1: 輝度=6.82, 重さ=1434g

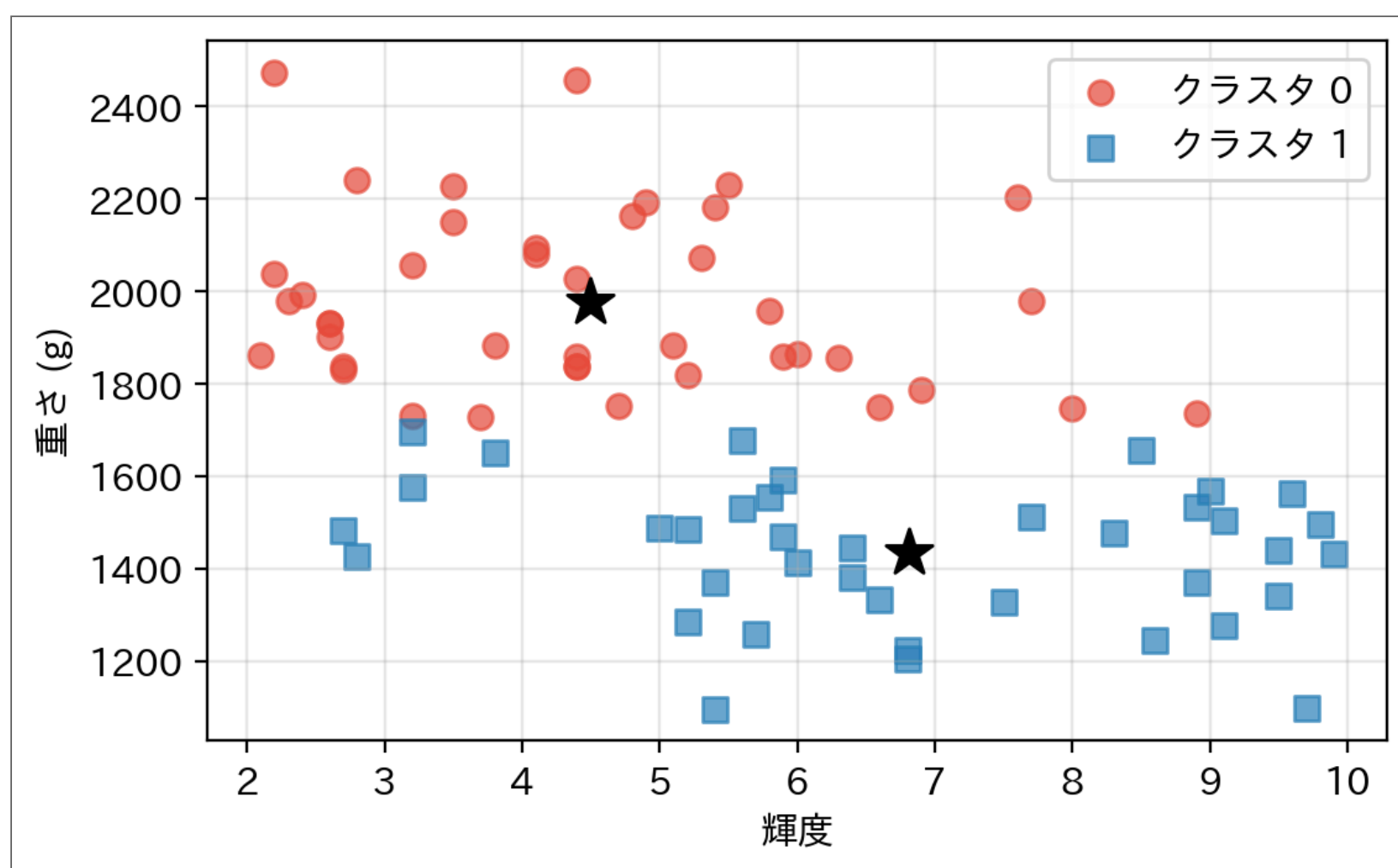
- エラーなく動いた。重心も2つ出た
- では結果を図でしてみる

8

試行1の結果

Code

```
1 colors = ["#e74c3c", "#2980b9"]
2 markers = ["o", "s"]
3
4 plt.figure(figsize=(5.8, 3.6))
5 for c in range(2):
6     mask = cluster_labels == c
7     plt.scatter(X[mask, 0], X[mask, 1],
8                 c=colors[c], marker=markers[c], s=50, alpha=0.7, label=f"クラスタ {c}")
9 for center in km.cluster_centers_:
10    plt.scatter(*center, c="black", s=200, marker="*", zorder=5)
11 plt.xlabel("輝度"); plt.ylabel("重さ (g)")
12 plt.legend(); plt.grid(True, alpha=0.3)
13 plt.tight_layout()
14 plt.show()
```



🚨 クイズ

目に見える塊は「左上」と「右下」。なのに機械の答えは**ほぼ真横の輪切り**。輝度を完全に無視しているのはなぜ？

9

なぜ輝度が無視されたのか

- k-means は**距離**だけでグループを決める
- 重さの差は**数百 g**、輝度の差は**せいぜい数ポイント**

$$\text{距離}^2 = \underbrace{(\Delta \text{輝度})^2}_{\sim 10} + \underbrace{(\Delta \text{重さ})^2}_{\sim 100,000}$$

- 距離の計算が**重さに支配され**、輝度はあってもなくても同じだった

⚠ スケールの罠

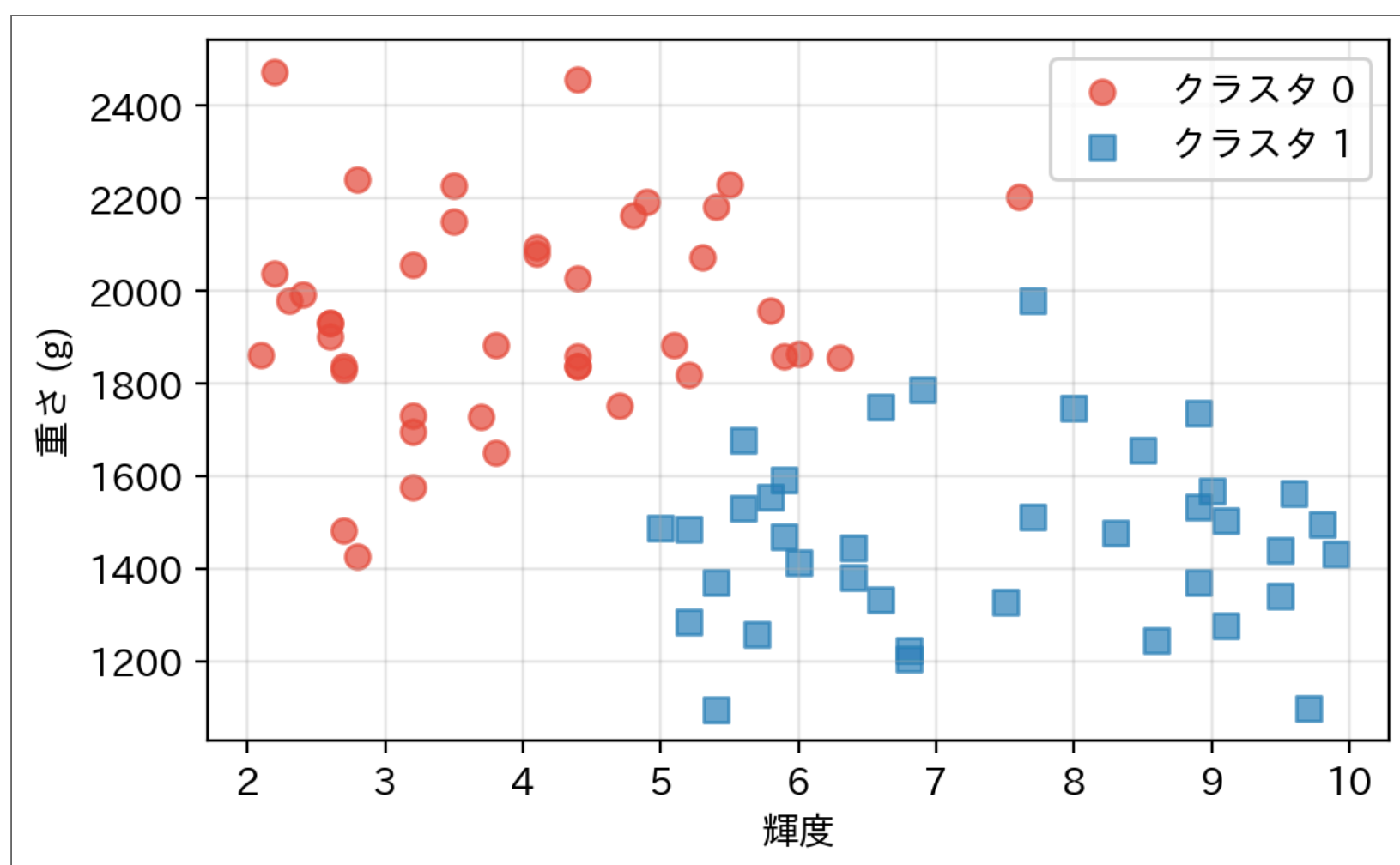
- 単位もケタも違う特徴量をそのまま距離計算に入れてはいけない
- 距離を使う手法（k-means、k-NN など）共通の落とし穴

10

試行2: スケールを揃えて再挑戦

Code

```
1 from sklearn.preprocessing import StandardScaler
2
3 X_scaled = StandardScaler().fit_transform(X) # 各特徴量を平均0・分散1に
4
5 km2 = KMeans(n_clusters=2, random_state=42, n_init=10)
6 km2.fit(X_scaled)
7 cluster_labels2 = km2.labels_
8
9 plt.figure(figsize=(5.8, 3.6))
10 for c in range(2):
11     mask = cluster_labels2 == c
12     plt.scatter(X[mask, 0], X[mask, 1],
13                 c=colors[c], marker=markers[c], s=50, alpha=0.7, label=f"クラスタ {c}")
14 plt.xlabel("輝度"); plt.ylabel("重さ (g)")
15 plt.legend(); plt.grid(True, alpha=0.3)
16 plt.tight_layout()
17 plt.show()
```



- 今度は **目で見たと通りの2つの塊**に分かれた

答え合わせ：実はラベルを知っていた

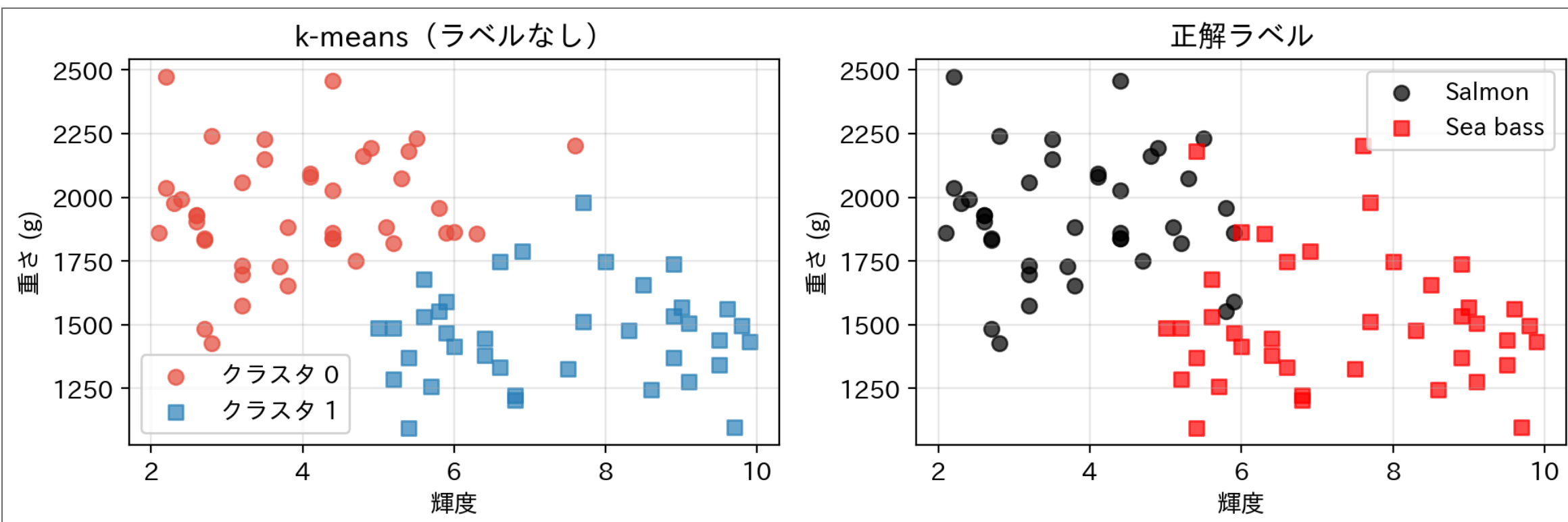
- このデータの正体は、これまでの講義で使ってきた Salmon / Sea bass

Code

```
1 y_true = np.array([1]*40 + [0]*40) # 1=Salmon, 0=Sea bass (隠していた正解)
2
3 agree = np.mean(cluster_labels2 == y_true)
4 agree = max(agree, 1 - agree) # クラスタ番号 0/1 は任意なので向きを合わせる
5 print(f"正解ラベルとの一致: {agree:.1%} ({int(round(agree * len(y_true)))}/80匹)")
```

Result

正解ラベルとの一致: 92.5% (74/80匹)

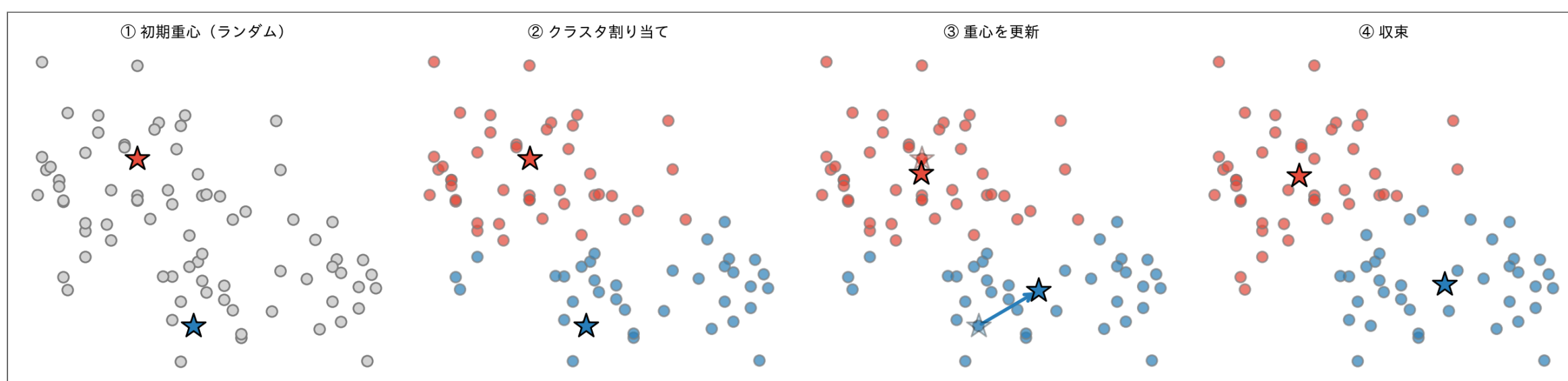


① ノート

ラベルを **1枚も見せていない** のに、80匹中74匹を正しく仕分けた。 クラスタの **番号 (0/1)** は **任意** で、大切なのは「同じグループにまとまっているか」。

12

k-means の動き (中で起きていたこと)

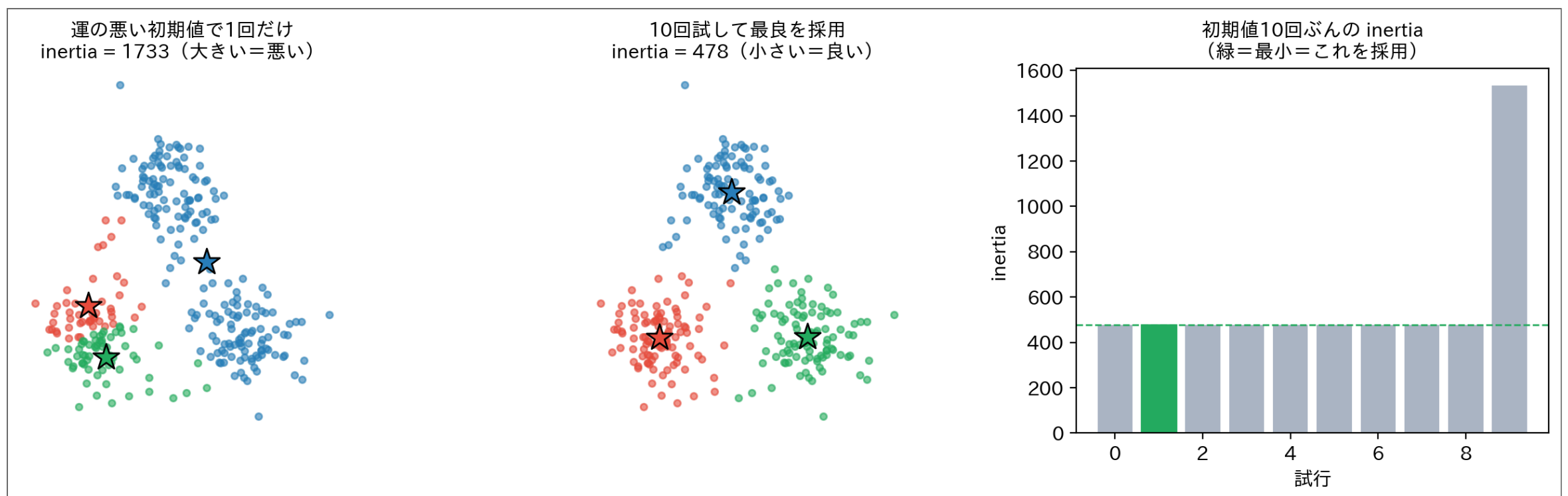


- どこに重心を置いても「assign → update」の繰り返しで塊に吸い寄せられていく
- スケールを揃えた空間 (標準化済み, **standardization**) での動き

13

ところで、最初の重心はどこに置く？

- 「assign → update」は**最初の重心の位置**から始まる — そこは**ランダム**に選ばれる
- 運の悪い初期値だと、おかしい所で落ち着く（下の左：青が2つの塊を飲み込んだ）



- k-means は「**クラスタ内二乗和 (inertia)** を小さくする」最適化 — でも初期値しだいで**悪い谷**にハマる
- 対策は「**何度も別の初期値で試して、一番 inertia が小さいものを採用**」

💡 これ、どこかで見た

初期値しだいで結果が変わる → 何回も試して最良を選ぶ。この作戦は**マルチスタート**と呼ばれ、参考資料 [ローカルミニマムの罠](#) で詳しく扱っている。 `KMeans(..., n_init=10)` の `n_init` が「10回試す」の正体（既定でこうなっている）。

14

k の決め方：エルボー法

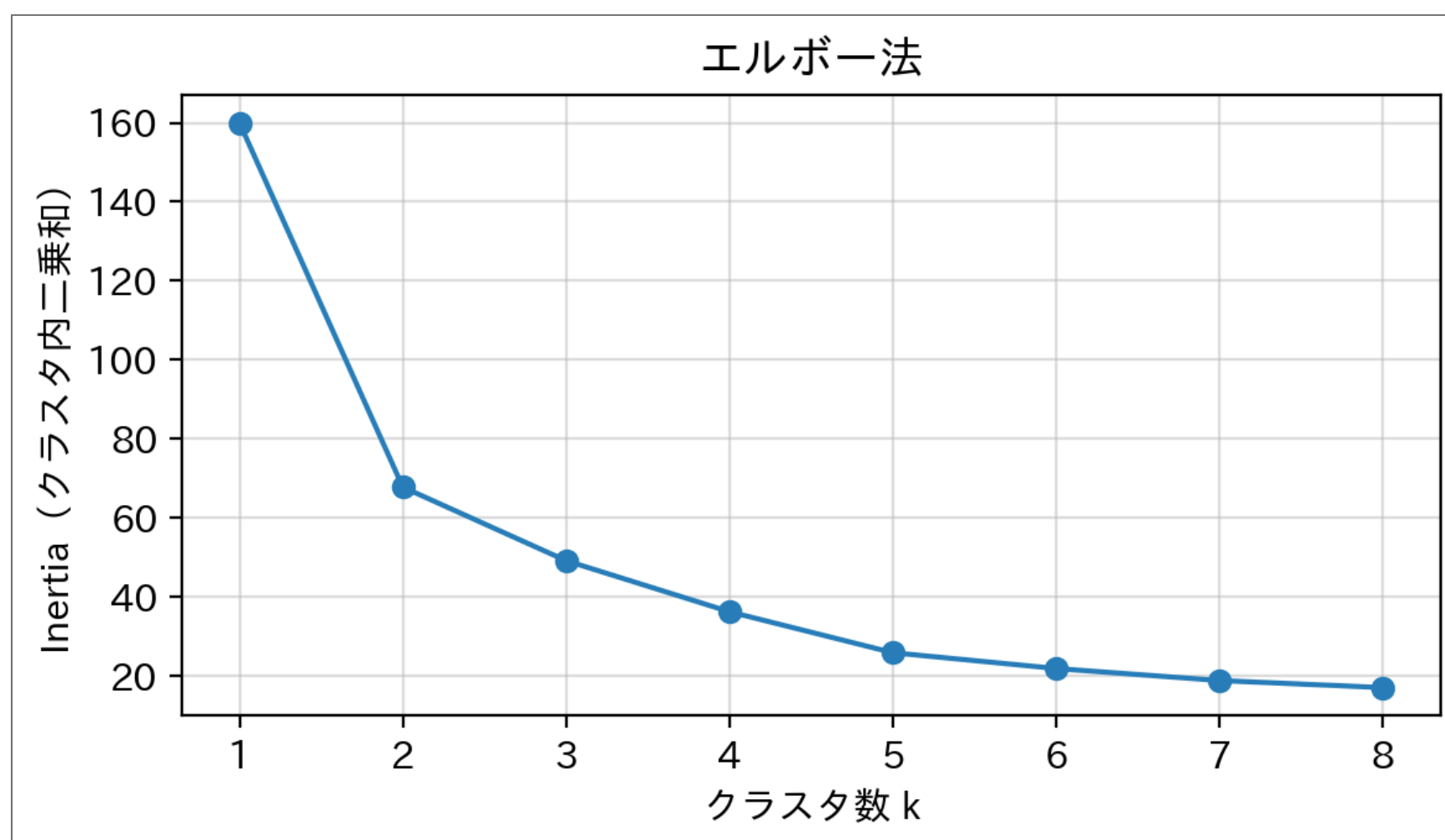
15

エルボー法

- 今回は「2種類の魚」と知っていたから $k=2$ にできた
- 本当のラベルなしデータでは **k 自体を決める必要がある**
- **エルボー法**：k を変えながら「クラスタ内のまとまり度」を測る

Code

```
1 inertias = []
2 ks = range(1, 9)
3
4 for k in ks:
5     km_k = KMeans(n_clusters=k, random_state=42, n_init=10)
6     km_k.fit(X_scaled)
7     inertias.append(km_k.inertia_)    # クラスタ内二乗和
8
9 plt.figure(figsize=(6, 3.5))
10 plt.plot(list(ks), inertias, "o-", color="#2980b9")
11 plt.xlabel("クラスタ数 k")
12 plt.ylabel("Inertia (クラスタ内二乗和)")
13 plt.title("エルボー法")
14 plt.grid(True, alpha=0.4)
15 plt.tight_layout()
16 plt.show()
```



💡 エルボー（肘）の見方

グラフが**急激に折れ曲がる点**（肘のような形）が最適な k の目安。それ以上増やしても改善幅が小さい。魚データでは $k=2$ で折れている。



- スケール済み k-means のコード を書き換えて実行（描画セルまで続けて実行）

1

クラスタを3つに

`n_clusters=2` → `3`（描画の
`range(2)` も `range(3)` に）

2

単位をキログラムに

`X_kg = X.copy(); X_kg[:, 1] /= 1000` を作って `km2.fit(X_kg)`

3

初期値ガチャを1回勝負に

`n_init=10` → `n_init=1`,
`init="random"`

💡 ここで観察

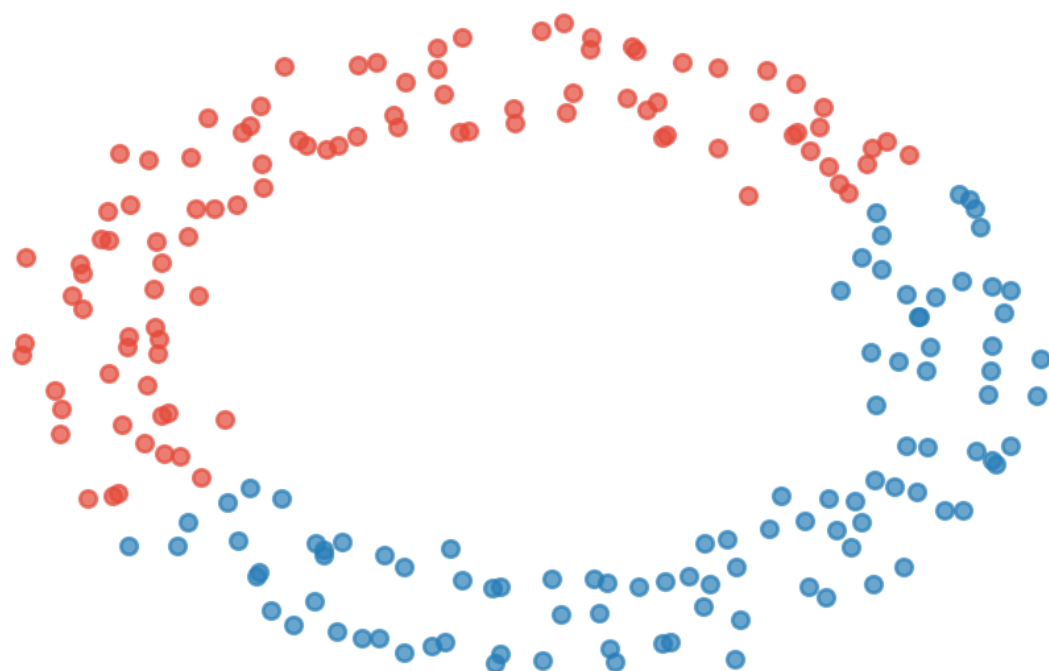
- $k=3$ の3つ目のクラスタは **Sea bass 側を割って** 生まれる（Salmon 34匹の塊はほぼ無傷）
— 増やした分は「一番ゆるい塊」が割られる
- グラム→キログラムに変えただけで、こんどは**輝度がほぼ独裁**して割り方が変わる — 「どの特徴量が偉いか」は**単位次第**。それを消すのがスケーリング
- この魚データでは1回勝負でも inertia は 67.8 前後でほぼ不動 — きれいな2山は初期値に強い（崩れる実例は「最初の重心」のスライドの3山データ）

17

k-means の限界

k-means は球状クラスタを仮定している

同心円（k-means は苦手）



三日月形（k-means は苦手）



⚠ k-means が苦手なケース

- 非球状のクラスタ（同心円、三日月形など）
- クラスタの**サイズが大きく異なる** 場合
- 初期値によって結果が変わる → `n_init`（マルチスタート）で回避（前述）

18

まとめ

- 教師なし学習：ラベルなしでデータの構造を見つける
- **k-means**：「assign → update → 繰り返し」だけで、魚80匹中74匹を正しく仕分けた
- **スケールの罠**：距離を使う手法では、特徴量のスケールを揃えてから

Code

```
1 from sklearn.cluster import KMeans
2 from sklearn.preprocessing import StandardScaler
3
4 X_scaled = StandardScaler().fit_transform(X) # まずスケールを揃える
5 km = KMeans(n_clusters=2, random_state=42, n_init=10)
6 km.fit(X_scaled)
7 km.labels_ # クラスタ番号（各データ点）
8 km.cluster_centers_ # 重心の座標
9 km.inertia_ # クラスタ内二乗和（エルボー法に使う）
```

長所	短所
シンプルで高速	k を事前に決める必要がある
結果が直感的	球状クラスタしか見つけれられない

① 次回予告

「特徴量を増やせば精度が上がった。では増やし続けたらどうなる？」

練習問題

問題1は**手計算**、問題2は**Pythonコード**で解け。

問題1：k-means を手で1周回す

4点だけの世界： $A(0, 0)$ 、 $B(0, 2)$ 、 $C(4, 0)$ 、 $D(4, 2)$ 。 $k = 2$ 、最初の重心を $M_1 = (0, 0)$ 、 $M_2 = (0, 2)$ に置く。

1. **assign**：各点を近い方の重心に割り当てよ（距離はユークリッド距離）
2. **update**：割り当てから新しい重心を計算せよ
3. assign が変わらなくなるまで繰り返せ。最終的なクラスタ分けと inertia（各点から自分の重心までの **二乗** 距離の合計）を求めよ
4. このデータの「自然な」分け方 $\{A, B\}$ と $\{C, D\}$ の inertia も計算し、3.の結果と比較せよ

問題2：n_init がこの罠を救うことを確かめる

1. **KMeans(n_clusters=2, init=np.array([[0.0, 0.0], [0.0, 2.0]]), n_init=1)** で問題1を再現し、**labels_** と **inertia_** を表示せよ
2. **KMeans(n_clusters=2, n_init=10, random_state=0)** で再実行し、**inertia_** を比較せよ

問題1の解答例

1周目 assign : $A \rightarrow M_1$ (距離0)、 $B \rightarrow M_2$ (距離0)、 $C \rightarrow M_1$ ($4 < \sqrt{20}$)、 $D \rightarrow M_2$ ($4 < \sqrt{20}$)

1周目 update : $M_1 = \text{mean}(A, C) = (2, 0)$ 、 $M_2 = \text{mean}(B, D) = (2, 2)$

2周目 assign : $A \rightarrow M_1$ ($2 < 2\sqrt{2}$)、 $B \rightarrow M_2$ 、 $C \rightarrow M_1$ 、 $D \rightarrow M_2$ — 変化なし、収束

結果 : $\{A, C\}$ と $\{B, D\}$ (上下の分割)。各点は重心から距離2なので

$\text{inertia} = 2^2 \times 4 = 16$

自然な分け方 $\{A, B\}, \{C, D\}$: 重心は $(0, 1)$ と $(4, 1)$ 、各点は距離1なので $\text{inertia} = 1^2 \times 4 = 4$ 。

❗ 手計算で罫にはまった

k-means の手順を **正しく** 回したのに、inertia 16 の **悪い答え** で止まった (最適は 4)。
assign → update は inertia を減らすことしかできず、**最初の重心の置き方** が悪いと抜け出せない。 本編の「変な分け方に落ち着く」を、4点で完全に再現できた。

問題2の解答例

▶ 解答例を見る

```
Result
|
| 悪い初期値: labels=[0 1 0 1], inertia=16.0
| n_init=10 : labels=[1 1 0 0], inertia=4.0
|
```

- 悪い初期値からは手計算どおり inertia 16 の分割に落ちる
- **n_init=10** は初期値を変えて10回回し、inertia 最小 (4) の分割を選んでくれる
- この「何回も試して一番良いのを選ぶ」作戦の一般論は、参考資料 [ローカルミニマムの罫](#) で